

Project Tinge

Navin

But what is a Path Tracer?

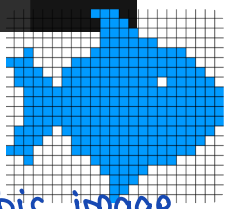


Wonder where the picture above was taken? Perhaps a NIKON DSLR? A Canon EOS maybe? Well, I took this image on an Intel i5 1135G7 processor. The word “took” is being used in a vague sense here. A more appropriate term that could be used is perhaps the word *render*. The image above is one that was entirely generated by a computer, from nothing but math and some elementary principles in optics.

The field of computer graphics has been rapidly evolving, with advancements in rendering techniques and algorithms pushing the boundaries of what’s visually possible. From the early days of flat, pixelated graphics we are now in an era where stunningly detailed, lifelike scenes are commonplace.

There are several techniques to model and render scenes in the digital world. *Rasterization* has been a foundational method since the early

taking an image described in vector graphics format (shapes) and converting into a raster image.



A raster graphic image

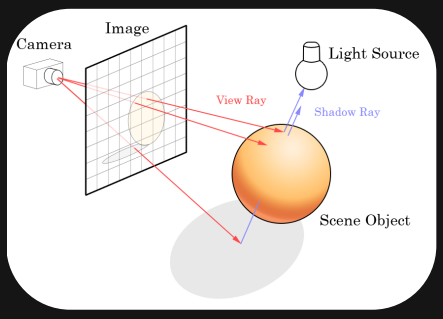
days of computer graphics. It involves projecting 3D objects onto a 2D screen – converting geometry into pixels. Rasterization became the standard in video games because it is fast and well-suited for real-time applications, even with limited hardware resources.

Path tracing, on the other hand, is a more recent development in rendering. I would say arguable, path-tracing is one of the most natural, physically-based, and straightforward ways of rendering images. Initially, path-tracing was impractical for real-time applications and was primarily used in offline rendering for animated movies (like Pixar's *RenderMan* engine). Recent advances in hardware and algorithms have started to bridge the gap, making ray-tracing (a simpler variant of path tracing) feasible in real-time scenarios as well (courtesy of Nvidia's RTX and similar video cards with dedicated computational cores).

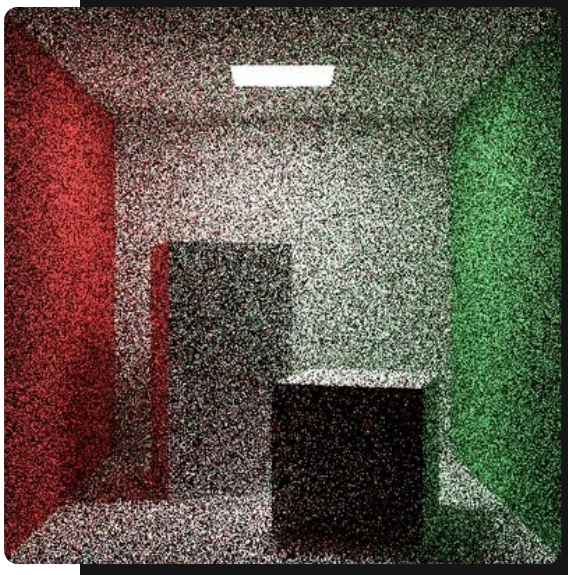
In due course of time, we will try to create a minimalist path-tracing engine supporting a robust scene system and a good variety of material and camera models. We will also be working on optimizing the performance of the engine at various levels—from probabilistic optimization techniques like importance sampling to micro-optimizations in code.

But how does it all work?

A schematic exhibiting the general workflow involved in a naive path-tracer is shown alongside (Image Credits: Wikipedia). The process involves back-tracking of finitely many rays from an observer's eye to the surrounding world (instead of tracking an infinitum of rays from a light source, hoping a few of them somehow reach the observer), modelling its interaction with the surface of intersection, computing irradiances, emittances and whatnot, tracing the ray forward, in essence accumulating a probabilistic measure of the energy that could have arrived from the actual forward-propagating ray from different light sources. Recent developments also incorporate a wave optics approach to more accurately model the behavior of light, consequently lending us more exciting effects like dispersion and lens aberrations.



Behind the scenes of fancy schematics lie some pretty heavy probability theory and linear algebra which are used in generating rays from the camera and filtering out noise and other artifacts that become prevalent when you solve the rendering equation for an arbitrary scene. In addition to this, several optimization techniques are often employed to accelerate the rendering process which typically takes minutes to generate a decent-looking image on a typical consumer-end device.



The image on the left is a brute-force low-sample render of the famous Cornell Box. Done incorrectly, your path-tracer might produce such somewhat hideous looking images.

The math involved in generating such scenes of wonder, is in itself quite a beauty. Let me not ramble too much and instead give you a starting point to work with. In this article, you will find tasks that can be broadly categorized into three sets – geometric algorithms, linear algebra and probability.

But how do I *model* shapes?

Before thinking about modelling shapes, it becomes imperative for us to first think of modelling rays. The simplest* model of a ray is the following:

$$R(t) = \vec{o} + t \times \vec{d},\ t \geq 0, \vec{o}, \vec{d} \in \mathbb{R}^3$$

where $\vec{o}$ can be thought of as the origin of the ray and $\vec{d}$ as a normalized direction. This model of a ray is sufficient for generating static images in a path-tracing engine. The parameter t is of considerable importance and is very useful in gauging the interaction of a ray with the surrounding shapes.

(*I call it simplest here, because in modern computer graphics, a ray often encapsulates much more information in order to capture effects such as motion-blur, absorption. Feel free to read up on this with a simple google search on "ray-tracing motion-blur")

Your tasks for the coming days

1. Come up with a procedure to find whether a ray intersects with a sphere (parametrized with center $\vec{c} \in \mathbb{R}^3$ and radius $r \in \mathbb{R}$). If it does, could you also compute the normal, a tangent and bi-tangent pair at the point of intersection? An

① $(z-\vec{c})^2 = r^2 \qquad z = \vec{o} + t\vec{d}$

$(|\vec{o}+t\vec{d}-\vec{c}|)^2 = r^2 \Rightarrow t^2 + 2t(\vec{o}-\vec{c})\vec{d} + [(|\vec{o}-\vec{c}|)^2 - r^2] = 0 \rightarrow$ quad int

$\Delta \geq 0 \rightarrow$ yes

$\vec{n} = \frac{\vec{z}-\vec{c}}{|\vec{z}-\vec{c}|}$ ($\vec{z}$ is P.O.I) $t = \frac{\vec{p}\times\vec{n}}{|\vec{p}\times\vec{n}|}$ $\vec{bt} = \vec{n}\times\vec{t}$ $\Delta < 0 \to$ no

$\vec{p} \to$ arbitrary vector not parallel to $\hat{n}$.

interesting thing to ponder over is to think about how negative values of r would be handled by your procedure. -ve r means sphere is just inside out i.e normal direction is reversed.

2. Come up with a procedure to find whether a ray- intersects with a parametrized 3D plane (normal $\vec{n}$ and passing through a point $\vec{p}$, where $\vec{n}, \vec{p} \in \mathbb{R}^3$). If it does, could you also compute the normal, a tangent and bi-tangent pair at the point of intersection?

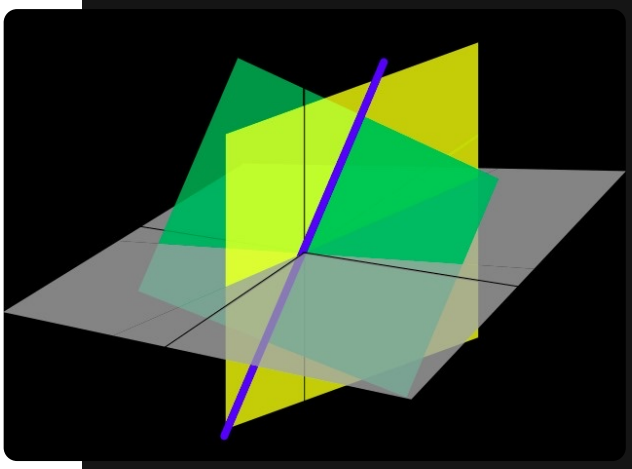
These tasks should hopefully get you comfortable with the kind of 3D geometry that is often invoked in computer graphics.

But how do I make things *richer*?

(2) $\vec{n}.(\vec{z}-\vec{p}) = 0 \Rightarrow \vec{n}.(\vec{o}+t\vec{d}-\vec{p}) = 0 \Rightarrow t = \frac{\vec{n}.\vec{p} - \vec{n}.\vec{o}}{\vec{n}.\vec{d}}$

$t = \frac{\vec{q}\times\vec{n}}{|\vec{q}\times\vec{n}|}$; $\vec{bt} = \vec{n}\times\vec{t}$

$= 0 \to$ parallel to plane ray is.



Often in computer graphics, we represent complex shapes in terms of primitives such as triangles or quadrilaterals. These primitives are captured by storing the position vectors of their vertices. In our path-tracer we would be playing around with various 3D models, moving them around, squishing and twirling them about to our hearts content.

The kinds of transformations that we work with come under the class of affine transformations – transformations that preserves parallelism and lines, but not necessarily Euclidean angles and distances.

These transformations are also useful when you wish to model a movable camera with which you wish to traverse your digital scene.

Your tasks for the coming days

1. Watch this https://www.3blue1brown.com/topics/linear-

algebra playlist (if you haven't already :)) to develop a general intuition on how matrices are used to represent linear transformations.

2. Go through this article https://brilliant.org/wiki/affine-transformations/ which gives you some examples of affine transformations.
3. Say I have a *position* vertex $\vec{v} = (0, 2, 4)$ which I wish to first scale by a factor of 2 along the axis $(1/2, 1/2, 1/\sqrt{2})$ and then move it by (1, 0, 1). What would be the final transformed vertex? Represent the transformation as an augmented matrix which incorporates translation of the vertex as well.

$v_{proj} = 0 \cdot 1/2 + 2 \cdot 1/2 + 4 \cdot 1/\sqrt{2} = (1+2\sqrt{2})(1/2, 1/2, 1/\sqrt{2})$

These tasks should get you up to speed on some useful elementary linear algebra concepts.

(3) $v_{scale} = 2 \times (1/2+\sqrt{2},\ 1/2+\sqrt{2},\ 1/\sqrt{2}+2) = (1+2\sqrt{2},\ 1+2\sqrt{2},\ 4+\sqrt{2})$

$v_{\perp} = (0,2,4) - (1/2+\sqrt{2},\ 1/2+\sqrt{2},\ 1/\sqrt{2}+2) = [-1/2-\sqrt{2},\ 3/2-\sqrt{2},\ 2-1/\sqrt{2}]$

But how do I *let there be light*?

$v = (1/2+\sqrt{2},\ 5/2+\sqrt{2},\ 6+1/\sqrt{2})$

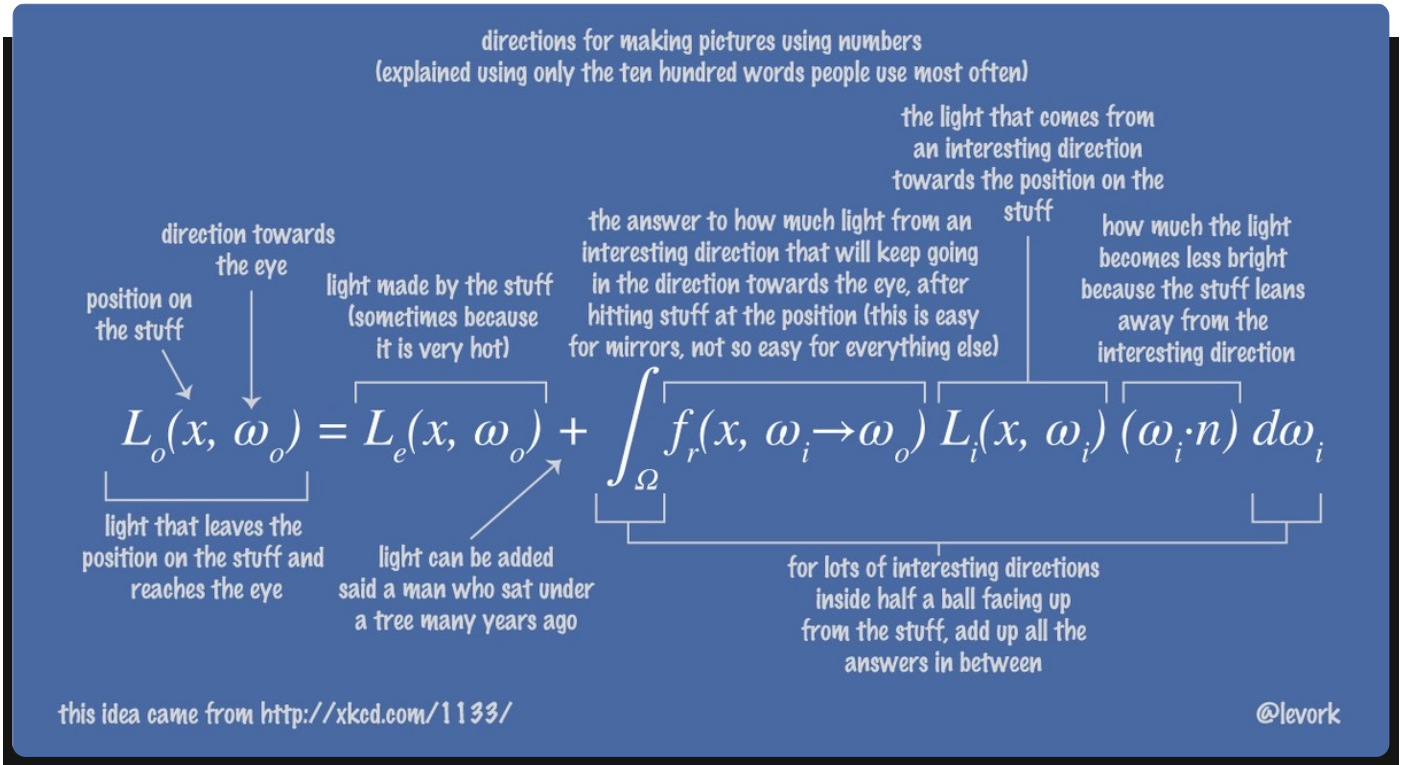
To incorporate lighting and material effects into your scene, what you must do is to solve the rendering equation, shown in the image below.

$v_{final} = (3/2+\sqrt{2},\ 5/2+\sqrt{2},\ 7+1/\sqrt{2})$

S=I+(scale factor−1) · (Normalized Axis) · (Normalized Axis)^T

Augmented matrix =

$$\begin{bmatrix} 5/4 & 1/4 & 1/2\sqrt{2} & 1 \\ 1/4 & 5/4 & 1/2\sqrt{2} & 0 \\ 1/2\sqrt{2} & 1/2\sqrt{2} & 3/2 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



This equation is something that we will encounter often and something that we would be approximating for various materials using certain heuristics. Often what is employed in solving the integral in this equation is a Monte Carlo Integrator which has its roots deep in probability theory and sampling techniques.

Your tasks for the coming days

1. Get comfortable with the ideas of probability distributions (both continuous and discrete) and the concepts of expectation, variance, PDFs and CDFs. Can you come up with intuitive explanations for these concepts? Feel free to refer to this brilliantly structured course from MIT OCW.

2. Try to work out the following problems:
    - Say X is a random variable with support $x > 0$ with a PDF $\propto \frac{1}{1+x^2}$. Can you compute the PDF and CDF of X?
    - What would be the variance of a probability distribution with the PDF given by $f(x) \propto e^{-x^2} \; \forall x \in \mathbb{R}$? Approximately how many standard deviations away from the origin do you think around 99% of the samples taken from this distribution would lie in?
3. Read up on how a Monte Carlo Estimator can be used to approximate the value of π and write a simple script in a language of your choice which implements the same. If you wish to dive deeper, maybe look into the idea behind importance sampling and develop an intuition as to how it speeds up the estimation process.

These tasks should hopefully get you comfortable with the some elementary probability concepts that would be handy in the mini-project.

But I want to do more!

If you find the work so far intriguing and wish to learn more

about / get more tasks to work on, then feel free to reach out to me. Don't hesitate to approach me in case you have any doubts with the tasks given above!

– Navin

© 2024 Mathematics Club IITM